

## Assignment 15 Git and GitHub

**Question 1) Explain what version control is and its importance in software development.**

**Ans)**

- A Version Control System (VCS) is a tool that helps manage changes to source code over time.
- It keeps track of every modification made to the code, allowing multiple people to collaborate on a project.
- VCS ensures that changes can be tracked, reviewed, and, if necessary, rolled back.

### **Importance in Software Development**

- **Collaboration:** Enables multiple developers to work on the same project simultaneously without conflicts.
- **History Tracking:** Keeps a record of all changes, who made them, and when they were made.
- **Backup and Restore:** Facilitates recovery of previous versions if something goes wrong.
- **Branching and Merging:** Allows developers to work on new features or fixes in isolated branches and merge them when ready.

**Question 2) Explain the Git Workflow, including the staging area, working directory, and repository**

**Ans)**

- **Working Directory** → Where you create/modify files.  
(Untracked/Modified state)
- **Staging Area** → A place to mark selected changes (git add) before committing.
- **Repository (Local)** → Permanent history of commits (git commit).
- **Remote Repository** → Shared repo (e.g., GitHub) where you push/pull changes.

**Question 3) Explain what .gitignore is and why it's important in version control.**

**Ans)**

A **special file** in Git that lists patterns for files and folders Git should **ignore** (not track).

Example:

```
*.log  
node_modules/  
.env
```

- Prevents committing **unnecessary files** (e.g., logs, temporary files, build artifacts).
- Protects **sensitive information** (like passwords or API keys in .env).
- Keeps the repository **clean and lightweight**, focusing only on source code and important files.

In short: **.gitignore helps maintain a clean, secure, and efficient Git repo by excluding unwanted files.**

**Question 4) Briefly explain what GitHub is and how it facilitates collaboration and version control also name some alternatives to GitHub.**

**Ans)**

GitHub is a cloud-based platform that hosts Git repositories. It provides tools for version control, allowing developers to track changes in their code, roll back to earlier versions, and manage contributions from multiple people.

For collaboration, GitHub offers features like pull requests, code reviews, issue tracking, and project boards, making it easy for teams to work together on the same project without overwriting each other's work.

**Alternatives to GitHub include:**

- GitLab
- Bitbucket
- SourceForge
- AWS CodeCommit
- Azure Repos

Question) Describe the process of contributing to any open-source project on GitHub in a step-by-step manner.

Ans) Steps

### 1. Find a Project

- Browse GitHub or platforms like *Awesome Projects*, *First Contributions*, or *Good First Issues*.
- Choose a project that matches your interest and skill level.

### 2. Fork the Repository

- On the project's GitHub page, click Fork (top-right corner).
- This creates a personal copy of the repository under your account.

### 3. Clone the Fork

- Copy your fork to your local machine:

```
git clone https://github.com/your-username/project-name.git
```

Move into the project folder:

```
cd project-name
```

### 4. Set Upstream Remote

- Connect your fork with the original repository (so you can sync updates later):

```
git remote add upstream https://github.com/original-owner/project-name.git
```

### 5. Create a New Branch

- Always work on a separate branch, not main. Example:

```
git checkout -b feature-branch
```

### 6. Make Changes

- Edit code, fix bugs, or add features.
- Follow the project's contribution guidelines (check CONTRIBUTING.md or README.md).

### 7. Commit Your Changes

- Stage and commit changes:

```
git add .
```

```
git commit -m "Brief description of changes"
```

### 8. Push Changes to GitHub

- Push your branch to your fork:

`git push origin feature-branch`

#### 9. Create a Pull Request (PR)

- Go to your fork on GitHub.
- Click **Compare & pull request**.
- Write a clear description of your changes and submit the PR to the original repository

#### 10. Review and Merge

- Project maintainers will review your PR.
- They may request changes. Update your branch, push again, and the PR updates automatically.
- Once approved, maintainers merge it into the project.

**Question 6) Deploy Tailwind projects named Youtube, slack, and Gmail clones on GitHub pages and share the deployed link of those three. Expected output - Live hosted URL Link of your deployed respective website with GitHub pages.**  
**\*Note - For this task, you can use HTML and CSS for now. Tailwind CSS is not required at this stage**

Answer)

Gmail Clone: <https://siddhant-kumar-04.github.io/gmail-clone>

Slack Clone: <https://siddhant-kumar-04.github.io/slack-clone>

Youtube Clone:  
<https://siddhant-kumar-04.github.io/youtube-clone>

